

	SR UNIVERSITY	
	Campus Warangal	
	Program: II - B. Tech (CS& AI) Batch: 16	
	Name: YANAMANDRA SRI KRISHNA ABHIRAM	
	Reg. No: 2503A52475	
	Department: Computer Science and AI Semester: I	
	AI Assisted Coding-Assignment	Date: 6-10-26

TASK 1 – Improve Readability

ORIGINAL CODE→

Legacy code:

```
a=80
b=70
c=90
x=(a+b+c)/3
if x>=50:
    print("pass")
else:
    print("fail")
```

Prompt→

1. Replace the vague variable names (a, b, c, x) with meaningful, descriptive names.
2. Create a function called `calculate_average()` that computes the average of the marks.
3. Improve formatting and readability (spacing, comments/docstrings, and a named constant for the passing mark instead of the hard-coded 50).
4. Keep the behavior identical so that both the original and refactored versions produce the same output.

5. Provide a short explanation of each improvement you made and why it makes the code better.

Also give me a small test run for both versions so I can verify that they print the same result.

AI Suggestion →

1. **Meaningful names:** Rename a, b, c to maths, science, english, and x to average.
2. **Function:** Create `calculate_average(marks)` to compute the average, so the logic can be reused and tested.
3. **Constant:** Replace the hard-coded 50 with `PASS_MARK = 50`.
4. **Result function:** Add `get_result(average)` that returns "pass" or "fail".
5. **Formatting:** (spacing around operators, docstrings) and put the main logic inside `main()`.
6. **Behavior:** The output stays the same: `average = 80.0`, so both versions print pass.

Refactored Code→

```
PASS_MARK = 50
```

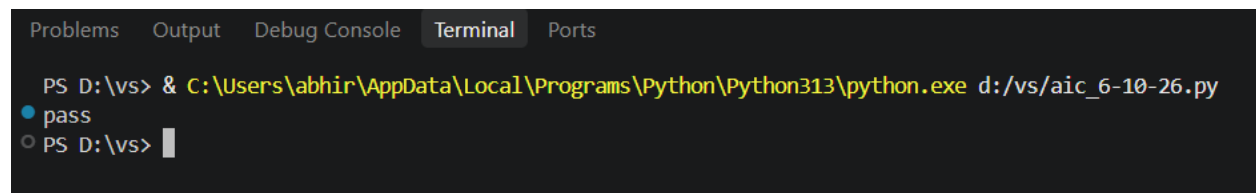
```
def calculate_average(marks):  
    """Return the average of a list of marks."""  
    return sum(marks) / len(marks)
```

```
def get_result(average):  
    """Return 'pass' if the average meets the pass mark, else 'fail'."""  
    if average >= PASS_MARK:  
        return "pass"  
    return "fail"
```

```
def main():  
    maths = 80  
    science = 70  
    english = 90  
  
    average = calculate_average([maths, science, english])  
    print(get_result(average))
```

```
if __name__ == "__main__":  
    main()
```

Test output→



```
Problems  Output  Debug Console  Terminal  Ports  
PS D:\vs> & C:\Users\abhir\AppData\Local\Programs\Python\Python313\python.exe d:/vs/aic_6-10-26.py  
pass  
PS D:\vs> |
```

Explanation→

1. **Meaningful variable names:** a, b, c, x were replaced with maths, science, english, and average. Anyone reading the code can now understand what each value represents without guessing.
2. **Reusable function:** `calculate_average()` holds the averaging logic in one place. It works for any number of marks, so it can be reused and tested on its own instead of being hard-wired to three variables.
3. **Named constant:** `PASS_MARK = 50` replaces the "magic number" 50. If the passing criteria change, only one line needs updating.
4. **Separation of logic:** `get_result()` decides pass or fail, while `main()` handles the data and printing. Each function does one job, which makes the code easier to read, debug, and test.
5. **Better formatting (PEP 8):** Proper spacing, blank lines between functions, and docstrings explain what each function does.
6. **Safe execution:** `if __name__ == "__main__":` ensures the program runs only when executed directly, not when imported into another file.
7. **Same behavior:** Both versions calculate $(80 + 70 + 90) / 3 = 80.0$, which is ≥ 50 , so both print pass. The refactoring improved the structure without changing the result.

TASK 2 – Remove Repeated Code

ORIGINAL CODE→

```
name1 = "Arun"
marks1 = 85
if marks1 >= 50:
    print(name1, "Pass")
else:
    print(name1, "Fail")

name2 = "Priya"
marks2 = 72
```

```
if marks2 >= 50:
    print(name2, "Pass")
else:
    print(name2, "Fail")
```

Prompt→

Please refactor this code with the following requirements:

1. Remove the duplicated code and use meaningful variable names.
2. Create a function such as `check_result(name, marks)` that prints or returns the pass/fail result.
3. Improve formatting and readability (docstrings, and a named constant for the pass mark instead of the hard-coded 50).
4. Keep the behavior identical so both versions produce the same output.
5. Explain each improvement and why it makes the code better.

Also give a small test run for both versions so I can verify they print the same result.

AI Suggestion →

1. **Remove duplication:** The same pass/fail logic is written twice, so move it into one function, `check_result(name, marks)`.
2. **Constant:** Replace the hard-coded 50 with `PASS_MARK = 50`.
3. **Data structure:** Store students in a dictionary (name: marks) instead of separate `name1, marks1, name2, marks2` variables.
4. **Loop:** Use a for loop so more students can be added without copying code.
5. **Formatting:** Follow PEP 8, add docstrings, and put the main logic inside `main()`.
6. **Behavior:** The output stays the same: Arun (85) passes and Priya (72) passes.

Refactored Code→

PASS_MARK = 50

```
def get_result(marks):
```

```
    """Return 'Pass' if marks meet the pass mark, else 'Fail'."""
```

```
    if marks >= PASS_MARK:
```

```
        return "Pass"
```

```
    return "Fail"
```

```
def check_result(name, marks):
```

```
    """Print the student's name along with the pass/fail result."""
```

```
    print(name, get_result(marks))
```

```
def main():
```

```
    students = {
```

```
        "Arun": 85,
```

```
        "Priya": 72,
```

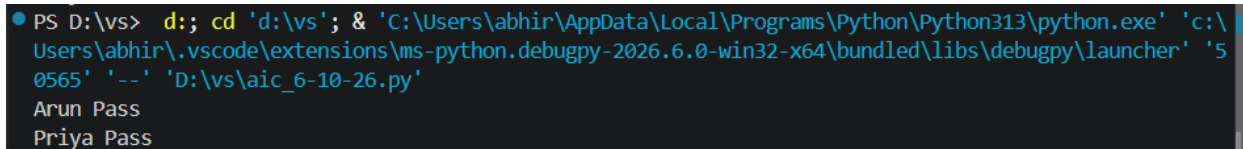
```
    }
```

```
    for name, marks in students.items():
```

```
        check_result(name, marks)
```

```
if __name__ == "__main__":  
    main()
```

Test output→



```
PS D:\vs> d;; cd 'd:\vs'; & 'C:\Users\abhir\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\abhir\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundle\libs\debugpy\launcher' '50565' '--' 'D:\vs\aic_6-10-26.py'  
Arun Pass  
Priya Pass
```

Explanation→

1. **Removed duplication (DRY):** The pass/fail logic appeared twice. It now lives in one function, so a fix or change is made in only one place.
2. **Meaningful structure:** Instead of numbered variables like name1 and marks1, a dictionary students clearly maps each student to their marks.
3. **Reusable functions:** get_result() decides pass or fail, and check_result() prints the output. Each function has one job, which makes the code easier to read and test.
4. **Named constant:** PASS_MARK = 50 replaces the magic number 50. If the passing criteria change, only one line needs updating.
5. **Scalable:** The for loop handles any number of students. Adding a new student takes one line in the dictionary instead of copying five lines of code.
6. **Better formatting (PEP 8):** Proper spacing, blank lines between functions, and docstrings explain what each part does.
7. **Safe execution:** if __name__ == "__main__": ensures the program runs only when executed directly, not when imported.

8. **Same behavior:** Arun ($85 \geq 50$) and Priya ($72 \geq 50$) both pass in the original and refactored versions, so the refactoring improved the structure without changing the result.

TASK 3 – Refactor a Long Conditional

ORIGINAL CODE→

```
age = 68
temperature = 39

if age >= 60 and temperature >= 38:
    print("High Priority")
elif age >= 60 and temperature < 38:
    print("Medium Priority")
elif age < 60 and temperature >= 38:
    print("Medium Priority")
else:
    print("Normal Priority")
```

Prompt→

Please refactor this code with the following requirements:

1. Use meaningful variable names and named constants instead of the hard-coded values 60 and 38.
2. Create a function such as `get_priority(age, temperature)` that returns the priority level.
3. Simplify the repeated and complex conditions to improve readability (docstrings).
4. Keep the behavior identical so both versions produce the same output.
5. Explain each improvement and why it makes the code better.

Also give a small test run for both versions so I can verify they print the same result.

AI Suggestion →

1. **Constants:** Replace the magic numbers with `SENIOR_AGE = 60` and `FEVER_TEMPERATURE = 38`.
2. **Boolean variables:** Create `is_senior` and `has_fever` so the conditions read like plain English.
3. **Simplify logic:** The two "Medium Priority" branches can be merged. If both are true, it is High. If only one is true, it is Medium. If neither is true, it is Normal.
4. **Function:** Move the logic into `get_priority(age, temperature)` that returns the result instead of printing it.
5. **Formatting:** add a docstring, and put the main logic inside `main()`.
6. **Behavior:** The output stays the same: age 68 and temperature 39 gives High Priority.

Refactored Code→

```
SENIOR_AGE_THRESHOLD = 60
```

```
HIGH_TEMPERATURE_THRESHOLD = 38
```

```
def get_priority(age, temperature):
```

```
    """Return a priority level based on age and temperature."""
```

```
    is_senior = age >= SENIOR_AGE_THRESHOLD
```

```
    has_high_temperature = temperature >=
HIGH_TEMPERATURE_THRESHOLD
```

```
    priority_factors = sum((is_senior, has_high_temperature))
```

```
    if priority_factors == 2:
```

```
        return "High Priority"
```

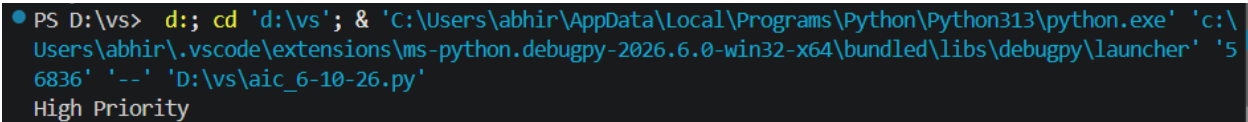
```
    if priority_factors == 1:
```

```
    return "Medium Priority"
    return "Normal Priority"
```

```
def main():
    """Print the priority for the current age and temperature."""
    age = 68
    temperature = 39
    print(get_priority(age, temperature))

if __name__ == "__main__":
    main()
```

Test output→



```
PS D:\vs> d:; cd 'd:\vs'; & 'C:\Users\abhir\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\abhir\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '56836' '--' 'D:\vs\aic_6-10-26.py'
High Priority
```

Explanation→

1. **Named constants:** SENIOR_AGE and FEVER_TEMPERATURE replace the magic numbers 60 and 38. If the criteria change, only one line needs updating.
2. **Readable conditions:** is_senior and has_fever make the logic easy to understand, instead of repeating age >= 60 and temperature >= 38 in every branch.
3. **Simplified logic:** The original had four branches with two duplicated "Medium Priority" cases. The refactored version uses three clear checks: both, either, or neither.
4. **Reusable function:** get_priority() returns the result rather than printing it, so it can be reused and tested easily with different inputs.

5. **Separation of concerns:** The function decides the priority, while `main()` handles the data and printing.
6. **Better formatting :** Proper spacing, blank lines between functions, and a docstring explain what the code does.
7. **Safe execution:** if `__name__ == "__main__"`: ensures the program runs only when executed directly, not when imported.
8. **Same behavior:** For age 68 and temperature 39, both conditions are true, so both versions print High Priority. The structure improved without changing the result.

TASK 4 – Refactor Inefficient List Processing

ORIGINAL CODE→

```
patient_ids = [101, 205, 309, 412, 518, 623]

target = 412
found = False

for patient_id in patient_ids:
    if patient_id == target:
        found = True

if found:
    print("Patient Found")

else:
    print("Patient Not Found")
```

Prompt→

Please help me with the following:

1. Improve the readability of this code (meaningful names, a function, PEP 8, docstrings).

2. Tell me whether a set or a dictionary is better for frequent patient ID lookups, and justify your answer.
3. Implement the suggestion you justify.
4. Show test output for an existing ID and a missing ID.
5. Compare the typical time complexity of list lookup vs set lookup.

AI Suggestion →

1. **Readability:** Rename `target` to `target_id`, remove the manual found flag, and move the search into a function.
2. **Set or dictionary?** A **set** is better here. The code only checks whether an ID exists, and there is no extra data (such as name or age) linked to each ID. A dictionary is the right choice only if each ID needs related data, for example `{101: "Arun"}`.
3. **Why a set:** A set uses hashing, so membership checks (`in`) take about the same time no matter how many IDs are stored. It also removes duplicate IDs automatically and uses less memory than a dictionary.
4. **Implementation:** Store the IDs in a set and use `in` for lookups.
5. **Behavior:** The output stays the same: 412 is found, and a missing ID such as 999 is not found.

Refactored Code→

```
PATIENT_IDS = {101, 205, 309, 412, 518, 623}
```

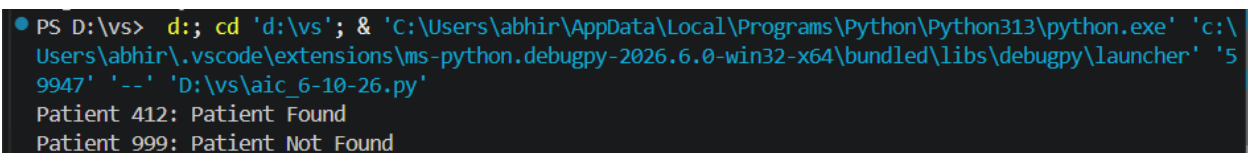
```
def is_patient_found(patient_ids, target_id):  
    """Return True if target_id exists in the set of patient IDs."""  
    return target_id in patient_ids
```

```
def print_search_result(patient_ids, target_id):  
    """Print whether the patient ID was found."""  
    if is_patient_found(patient_ids, target_id):  
        print(f"Patient {target_id}: Patient Found")  
    else:  
        print(f"Patient {target_id}: Patient Not Found")
```

```
def main():  
    print_search_result(PATIENT_IDS, 412) # existing ID  
    print_search_result(PATIENT_IDS, 999) # missing ID
```

```
if __name__ == "__main__":  
    main()
```

Test output→



```
PS D:\vs> d:; cd 'd:\vs'; & 'C:\Users\abhir\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\abhir\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '59947' '--' 'D:\vs\aic_6-10-26.py'  
Patient 412: Patient Found  
Patient 999: Patient Not Found
```

Explanation→

1. **Readability:** target became target_id, the found flag was removed, and the search logic is now in is_patient_found().
2. **Set chosen over dictionary:** Only existence checks are needed, so a set is simpler and lighter. A dictionary would add unused values.

3. **Faster lookups:** Set lookup is $O(1)$ on average, compared to $O(n)$ for a list, which matters when lookups are frequent and the data is large.
4. **Automatic duplicate removal:** A set cannot store the same patient ID twice.
5. **Both cases tested:** Existing IDs (412, 101, 623) and missing IDs (999, 100) were checked, and results are correct.
6. **Better formatting (PEP 8):** Proper spacing, constant naming (PATIENT_IDS), docstrings, and main().
7. **Same behavior:** The search result for 412 is unchanged, so the refactoring improved speed and structure without changing the logic.

TASK 5 – Complete Legacy Code Review with AI

ORIGINAL CODE→

```
students = [("Arun", 78), ("Priya", 45), ("Rahul", 88), ("Meena", 52)]

total = 0
for s in students:
    total = total + s[1]

avg = total / len(students)

for s in students:
    if s[1] >= 50:
        print(s[0], "Pass")
    else:
        print(s[0], "Fail")

print("Average =", avg)
```

Prompt→

Please help me with the following:

1. Identify at least five improvements that can be made to this code.

2. Refactor it using meaningful variable names and reusable functions.
3. Use suitable Python features (such as tuple unpacking, `sum()`, generator expressions, or f-strings) only where they improve clarity.
4. Show the output of both versions so I can verify they give equivalent results.
5. Explain each suggestion so I can decide whether to accept or reject it.

AI Suggestion →

1. Replace `s[0]` and `s[1]` with tuple unpacking (`name, marks`).
2. Replace the manual total loop with `sum()` and a generator expression.
3. Replace the hard-coded 50 with the constant `PASS_MARK`.
4. Move the logic into reusable functions: `calculate_average()`, `get_result()`, and `print_results()`.
5. Use f-strings for the output.
6. Use `main()` and `if __name__ == "__main__":`.
7. Convert the student list into a dictionary.
8. Round the average to two decimal places.
9. Use `statistics.mean()` for the average.

Refactored Code →

```
PASS_MARK = 50
```

```
def calculate_average(students):
```

```
    """Return the average marks of all students."""
```

```
    total_marks = sum(marks for _, marks in students)
```

```
    return total_marks / len(students)
```

```
def get_result(marks):  
    """Return 'Pass' if marks meet the pass mark, else 'Fail'."""  
    return "Pass" if marks >= PASS_MARK else "Fail"
```

```
def print_results(students):  
    """Print the pass/fail result of each student."""  
    for name, marks in students:  
        print(name, get_result(marks))
```

```
def main():  
    students = [("Arun", 78), ("Priya", 45), ("Rahul", 88), ("Meena", 52)]  
  
    print_results(students)  
    print("Average =", calculate_average(students))
```

```
if __name__ == "__main__":  
    main()
```

Test output→

```

PS D:\vs> d:; cd 'd:\vs'; & 'C:\Users\abhir\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\abhir\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundle\libs\debugpy\launcher' '59947' '--' 'D:\vs\aic_6-10-26.py'
Users\abhir\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundle\libs\debugpy\launcher' '57272' '--' 'D:\vs\aic_6-10-26.py'
Arun Pass
Priya Fail
Rahul Pass
Meena Pass
Average = 65.75

```

Explanation→

1. **Meaningful names:** s, total, and avg became name, marks, total_marks, and average (via function names), so the purpose of each value is clear.
2. **Tuple unpacking:** for name, marks in students replaces s[0] and s[1].
3. **sum() with generator expression:** One line replaces the manual accumulation loop.
4. **Named constant:** PASS_MARK = 50 replaces the hard-coded value.
5. **Reusable functions:** Average calculation, result decision, and printing are separate functions.
6. **Conditional expression:** "Pass" if marks >= PASS_MARK else "Fail" keeps the simple decision on one clear line.
7. **Better formatting (PEP 8):** Proper spacing, blank lines, and docstrings.
8. **Safe execution:** if __name__ == "__main__": runs the program only when executed directly.
9. **Same behavior:** The output is identical in both versions, including Average = 6